NurtureShield:

Machine Learning-Based Malicious URL and Phishing Detection System

CS3002 Information Security - Fall 2026
Term Project - Security Automation Tool

Abdul Rehman	**23L-0544**
Fahad Ahmed	**23L-0628**
Rayyan Kamran	**23L-0528**

1. Problem Statement

Malicious and phishing links are commonly used to steal credentials, redirect users to fake pages, or deliver harmful content. A user often has to decide whether to trust a link before opening it, when very little information is available. NurtureShield provides a quick machine-learning risk check from the URL text itself. The extension sends the link to a trusted backend, which analyzes its lexical and structural patterns without automatically visiting the submitted destination.

2. Features

- Browser-extension checking: paste a URL, use the current tab, or check a browser-provided hyperlink before choosing to open it.
- URL-only analysis: the suspicious destination is not automatically fetched, scraped, downloaded, or executed during classification.
- 31-feature ML analysis: the backend extracts a fixed set of lexical and structural URL features and produces a malicious probability.
- Clear risk result: the extension shows the probability, risk level, classification, and simple URL characteristics noticed by the system.
- Corrected URL representation: one leading HTTP/HTTPS scheme is removed before lexical feature calculation to avoid scheme-based shortcut learning.
- Protected API: input limits, rate limiting, trusted-client checks, safe error handling, and privacy-aware logging reduce service abuse.

3. Dataset

The model was developed using the Malicious URLs Dataset from Kaggle. It contains benign, phishing, defacement, and malware URLs. For this project, benign URLs are treated as legitimate (0), while phishing, defacement, and malware are combined as malicious (1). Duplicate/conflicting records were cleaned and domain-aware splitting was used so the same registered domains do not overlap between training, validation, and test data.

Dataset: https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset

Original class	Binary role
Benign	Legitimate (0)
Phishing	Malicious (1)
Defacement	Malicious (1)
Malware	Malicious (1)

3.1 Bias Correction and Feature Representation

During development, explicit URL schemes were strongly associated with some source classes. To reduce this shortcut, the corrected pipeline strips exactly one leading http:// or https:// before lexical feature calculation and removes the old uses_https feature. The final production representation contains 31 features. Tested bare, HTTP, and HTTPS variants later produced the same probability for the scheme-invariance examples.

4. System Architecture

The diagram below shows the complete analysis path. For the submitted product, the browser extension is the user-facing interface. A link is sent as text to the trusted API. The backend validates the request, canonicalizes the URL, extracts the fixed 31-feature vector, runs HistGradientBoosting predict_proba, and applies the 0.30 production threshold. A probability below 0.30 is reported as legitimate; a probability at or above 0.30 is reported as potentially malicious. Although the diagram also shows a web-interface entry point, this report focuses on the extension workflow.

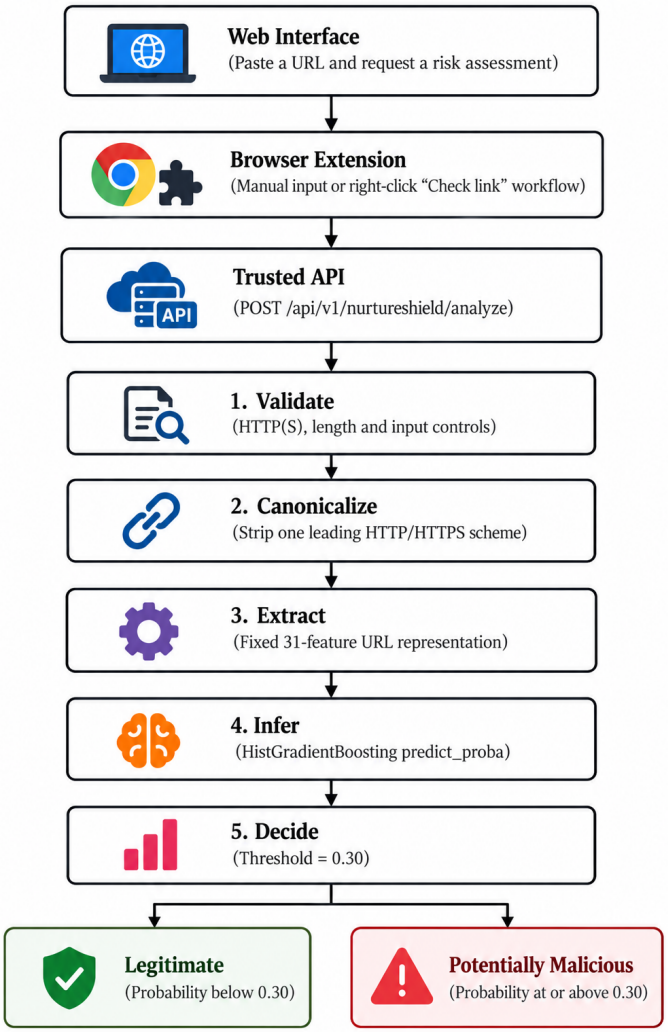


Figure 1. NurtureShield URL risk-assessment architecture and decision flow.

5. Extension Workflow

The extension is designed to help the user check a link before trusting it. It does not automatically browse to the suspicious destination.

- The user pastes a URL, chooses the current tab, or uses the browser's right-click NurtureShield action on a hyperlink.
- The extension sends only the URL text to the configured NurtureShield backend.
- The backend validates the request and runs canonicalization, 31-feature extraction, model inference, and the 0.30 production decision.
- The extension receives the malicious probability and displays a clear result with observed URL characteristics.
- The final choice to open or avoid the link remains with the user.

5.1 Security Design

Security is built around one rule: checking a suspicious link should not require visiting it. The extension treats the URL as untrusted text and the backend performs lexical/structural analysis only. The API validates and bounds input, rate-limits repeated requests, restricts trusted browser clients/origins, keeps secrets outside source code, avoids exposing internal errors, and avoids unnecessary logging of full URLs. The extension does not need to read arbitrary page content for the core check-link workflow. Results are advisory: a legitimate prediction reduces suspicion from the model's URL patterns, but it does not guarantee that the destination is safe.

6. Model Development and Selection

Three candidate classifiers were compared on validation data. Random Forest produced the strongest validation F1 and ranking scores in this comparison, while HistGradientBoosting produced the highest recall at its evaluated point. HistGradientBoosting was retained because this security use case prioritizes detecting malicious links and supports explicit threshold control.

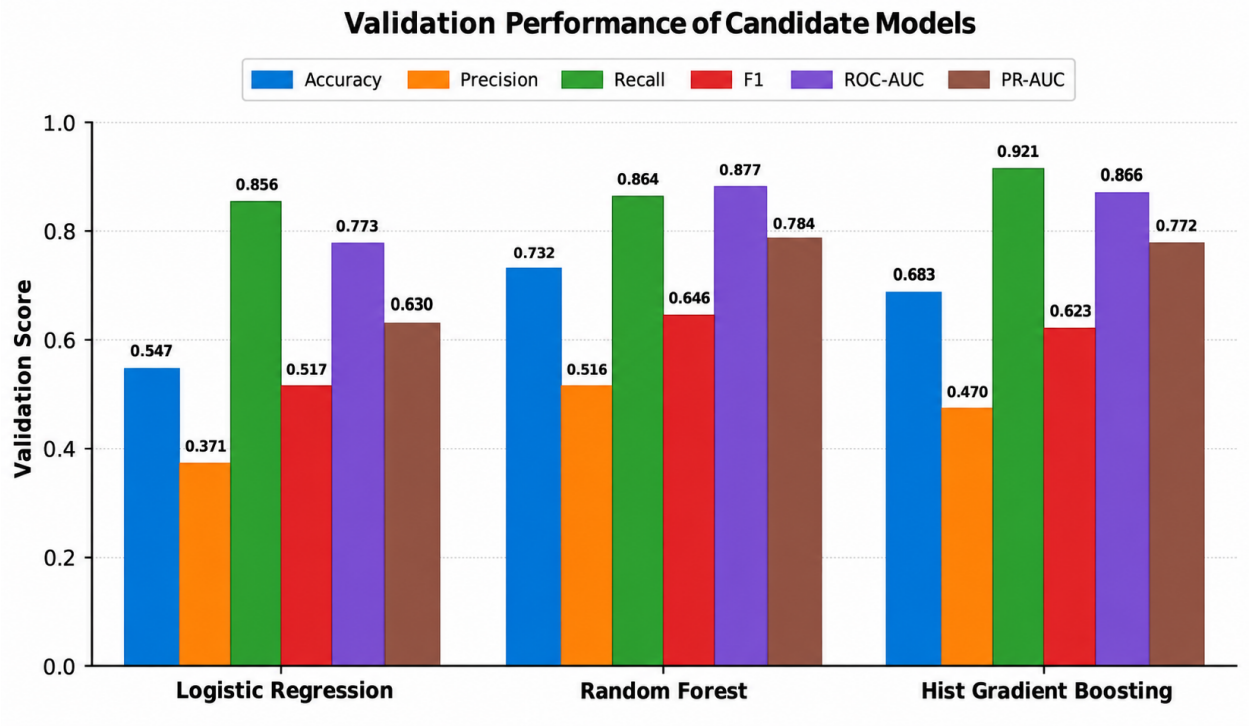


Figure 2. Validation performance of candidate models at their evaluated thresholds.

7. Production Threshold

The decision threshold controls the trade-off between catching more malicious links and incorrectly warning on legitimate links. The deployed threshold is 0.30, selected as a recall-oriented operating point. This production threshold is different from the 0.24 evaluation threshold used by the formal held-out test artifacts.

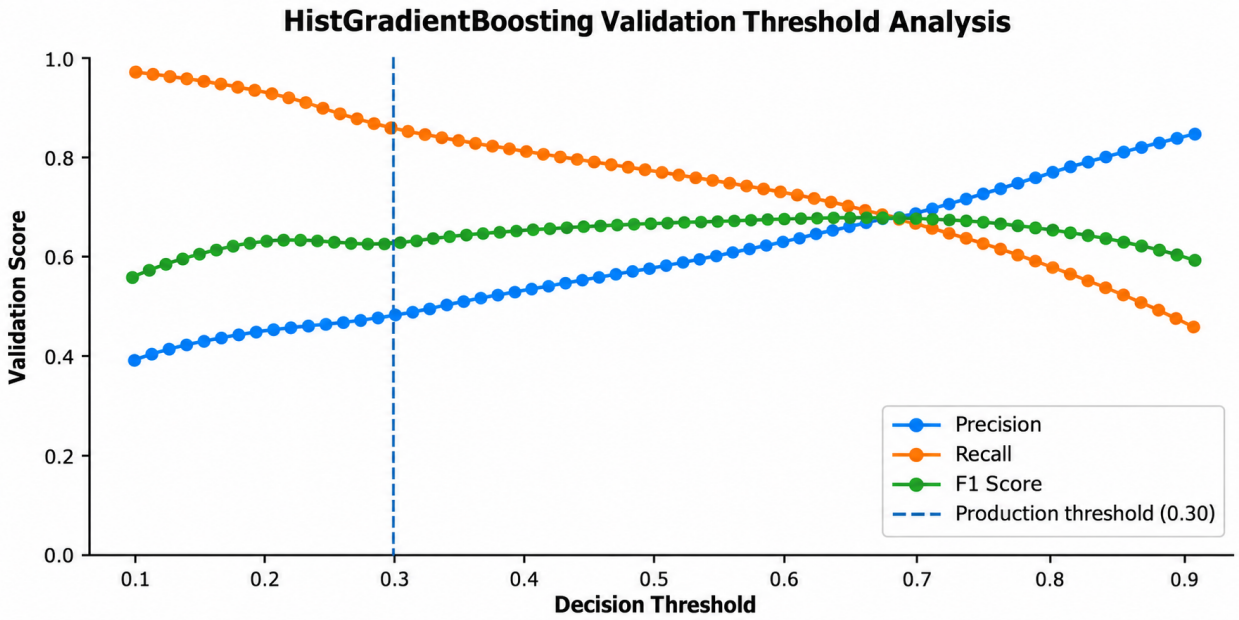


Figure 3. HistGradientBoosting validation threshold analysis; dashed line marks the 0.30 production threshold.

8. Experimental Results

8.1 Held-Out Test Performance

The formal corrected held-out test results below use evaluation threshold = 0.24. The values are shown as scores only, as requested.

Metric	Score
Accuracy	0.7477
Precision	0.6107
Recall	0.8930
F1 Score	0.7254
ROC-AUC	0.8777
PR-AUC	0.8385

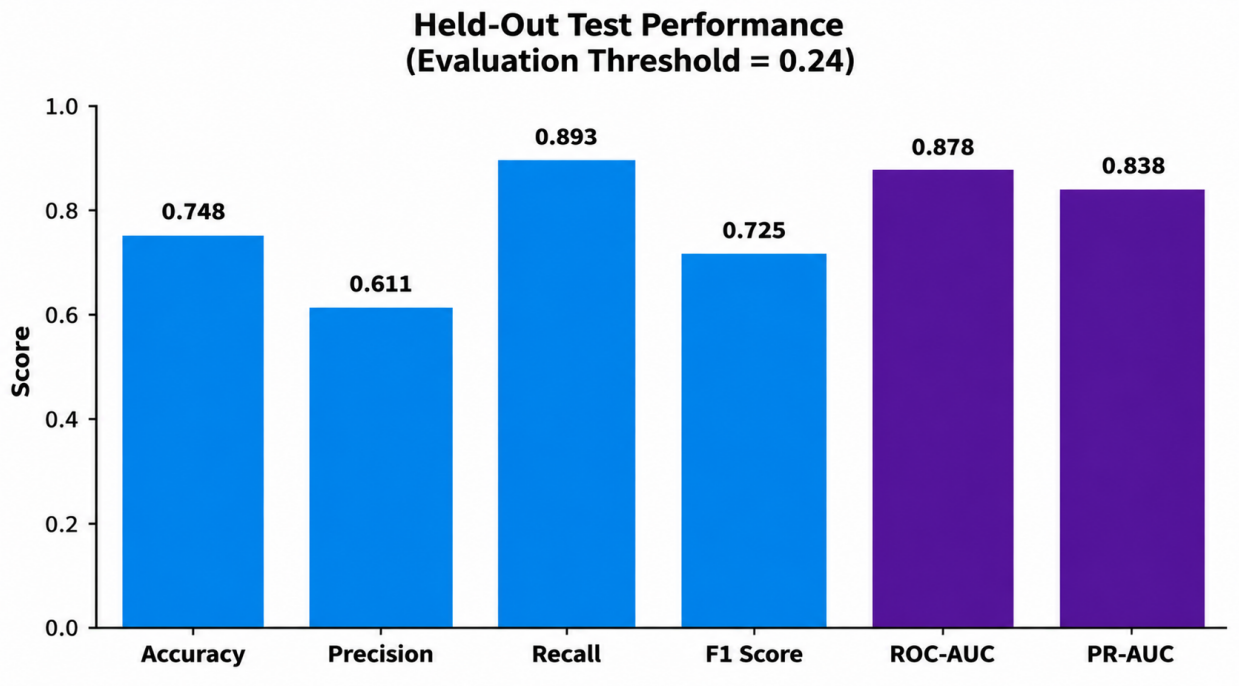


Figure 4. Held-out test performance at evaluation threshold 0.24.

8.2 Confusion Matrix

At threshold 0.24, the model produced 35,218 true negatives, 18,040 false positives, 3,390 false negatives, and 28,299 true positives. The result reflects the recall-oriented behavior of the model: most malicious URLs are detected, but more legitimate URLs are also warned about.

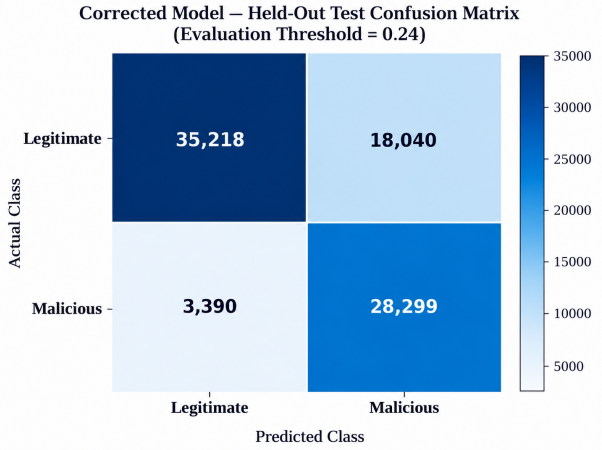


Figure 5. Held-out test confusion matrix at evaluation threshold 0.24.

8.3 Malicious Subtype Performance

Subtype	Samples	Recall	Mean malicious probability
Phishing	13,913	0.9069	0.613
Malware	3,602	0.6630	0.624
Defacement	14,174	0.9378	0.817

The model performs best on defacement and phishing URLs, while malware recall is lower. This is an important limitation because malicious URLs that look structurally ordinary can be harder for a lexical model to detect.

8.4 Robustness and Scheme Invariance

Transformation	Samples	Mean |Δ probability|	Prediction flips
Added Subdomain	5	0.209	1
Path Noise	5	0.063	0
Percent Encoding	5	0.046	0
Punycode	5	0.086	0
Query Noise	5	0.109	1
Scheme HTTP	5	0.000	0
Scheme HTTPS	5	0.000	0
Unicode	5	0.110	1

HTTP and HTTPS transformations caused zero probability change in the tested robustness set, supporting the scheme-canonicalization correction. Other transformations can still change the prediction, so these tests show remaining sensitivity rather than proving universal robustness.

9. Project Source Code

The implementation is referenced through the submitted source files rather than reproduced inside this report. Important backend files include the API entry point, the NurtureShield service/inference module, the 31-feature extractor, URL-characteristic generation, the trained model/configuration, and backend/security tests.

Backend repository: https://github.com/abdulrehman142/nurtureshield_backend.git

Key files: app.py; nurtureshield/features.py; nurtureshield/service.py; nurtureshield/characteristics.py; models/hist_gradient_boosting.joblib; models/final_model_config.json; tests/test_nurtureshield.py.

10. How to Run

10.1 Backend

```
git clone https://github.com/abdulrehman142/nurtureshield_backend.git
cd nurtureshield_backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
Configure the values in .env.example
uvicorn app:app --host 0.0.0.0 --port 8000
```

10.2 Browser Extension

- Download or clone the NurtureShield extension source used for the project.
- Open chrome://extensions in a Chromium-based browser.
- Enable Developer mode and choose Load unpacked.
- Select the folder containing manifest.json.
- Use the popup or right-click link workflow; the extension sends the URL to the configured deployed backend.

11. Findings and Limitations

- Dataset shortcuts matter: high apparent performance can be misleading when representation features such as URL scheme correlate with source classes.
- Recall matters in this use case: the held-out evaluation achieved a recall score of 0.8930, but this comes with false-positive cost.
- Thresholds must be stated clearly: 0.30 is the production operating threshold; 0.24 belongs to the formal held-out evaluation artifacts.
- URL-only analysis is safer but limited: the model cannot see destination page content, redirects, DNS behavior, certificates, or downloaded payloads.
- Malware remains harder: malware subtype recall is lower than phishing and defacement, leaving room for future improvement.

12. Conclusion

NurtureShield is a practical malicious-URL and phishing detection prototype centered on a Chromium browser extension. The extension lets a user check a link before deciding to open it, while the trusted backend performs corrected URL preprocessing, fixed 31-feature extraction, HistGradientBoosting inference, and threshold-based classification. The project combines machine-learning evaluation with security controls that reduce the risk of the checking process itself. Its output should be treated as a risk signal rather than a guarantee, but it provides a useful automated warning layer for suspicious links.